

SKYHOG: Density Based Low-Altitude Flight Planning Using A*

HUGH LONG, Florida State University, USA

DYLAN NEGVESKY, Florida State University, USA

Low altitude aerial surveys and pest control operations typically require a large amount of ground visibility in order to be successful. This can often present a challenge due to dense forest canopy, thick understory or large amounts of tall ground vegetation. You might suppose, then, that in order to facilitate more successful operations we might seek to maximize or at least reasonably improve the amount of ground visible while flying a given pattern over a search area. In the past, addressing this would have likely been functionally impossible – with no reliable access to high-quality forest density metrics covering a significant enough area. Recent proliferation and usage of LiDAR (Light Detection And Ranging) technology has made available a wealth of high resolution (>20 returns sq/m) data covering large swaths of the continental U.S.. In this paper we discuss a methodology to process and transform these 3-dimensional datasets into forms suitable for usage with common off-the-shelf pathfinding algorithms in order to maximize visible ground surface over a given flight area.

CCS Concepts: • **Computing methodologies** → **Planning and scheduling**.

Additional Key Words and Phrases: A* Algorithm, LiDAR, Flight Planning, Pathfinding, GIS

1 Introduction

Aerial surveys and pest control flights are often utilized in the U.S. Southeast for invasive species control as well as for general habitat management and data collection. These flights often run in a strip-wise manner across a designated area of operation, i.e. , from East to West or North to South. Given the scientific, habitat management or pest control goals of these flights, it is often desirable to maximize the amount of ground or near-ground surface visible from an aerial vehicle – like a helicopter. Advancements in and the proliferation of LiDAR usage across the U.S. has made available a wealth of data not previously accessible. LiDAR data typically takes the form of (x, y, z) points where x and y denote the coordinate position on Earth’s surface and z denotes the absolute height of the observation – typically referenced from sea level. These point clouds can be used to describe canopy density, demarcate structures, understand forest structure and a myriad of other tasks.

Pathfinding algorithms are a class of algorithms usually noted for their ability to minimize the cost of a path from a start position to a goal position. We can think of this as the minimization of an objective function that maps moves from one node in a network to another connected node in the network onto an element of the real numbers. A highly popular and performant pathfinding algorithm, A*, minimizes the cost of a path from a starting node to a goal node using a variation on the classic Dijkstra’s algorithm that introduces a heuristic function to reduce the size of the search space. A* is usually described by the following function, which describes the algorithm’s behavior at a given open node when evaluating its cost.

$$f(n) = g(n) + h(n) \quad (1)$$

Where $g(n)$ describes the distance from the starting node to the current node while $h(n)$ defines a heuristic function that estimates the cost to the goal node from the current node. There are quite a few valid $h(n)$, each with their own advantages. The only real requirement for $h(n)$ is that it must be consistent in that it must obey the following

Authors’ Contact Information: Hugh Long, Florida State University, Tallahassee, USA, hal20a@fsu.edu; Dylan Negvesky, Florida State University, Tallahassee, USA, dpn23@fsu.edu.

relationship when moving from a node n to a neighboring node m .

$$h(n) \leq c(n, m) + h(m) \quad (2)$$

Where $c(n, m)$ denotes the cost to move to m from n .

2 Methodology

Transforming 3D point clouds into a format compatible with a typical implementation of A* (and which may yield good results) requires a bit of finagling. While we could certainly model the problem in the 3D space using transformations on the point cloud, this would be prohibitive. The high resolution of these data means that even for medium areas of interest of only, say, $\approx 100\text{Km}^2$ we would be dealing with uncompressed datasets of around $\approx 50\text{GB}$. This is naturally far too much data to load and process every time we want to generate some flight lines – and it limits usage to only those who have access to high-quality hardware. To solve this, we can reduce the point cloud to a gridded 2D representation while still keeping most of the important features. This is a common operation in GIS workflows and is equivalent to creating a digital surface model (DSM) of the target area. A one meter starting resolution significantly compresses our data and provides plenty of detail for our purposes, while still maintaining the option to reduce resolution on the fly during computation if a lower resolution is desirable.

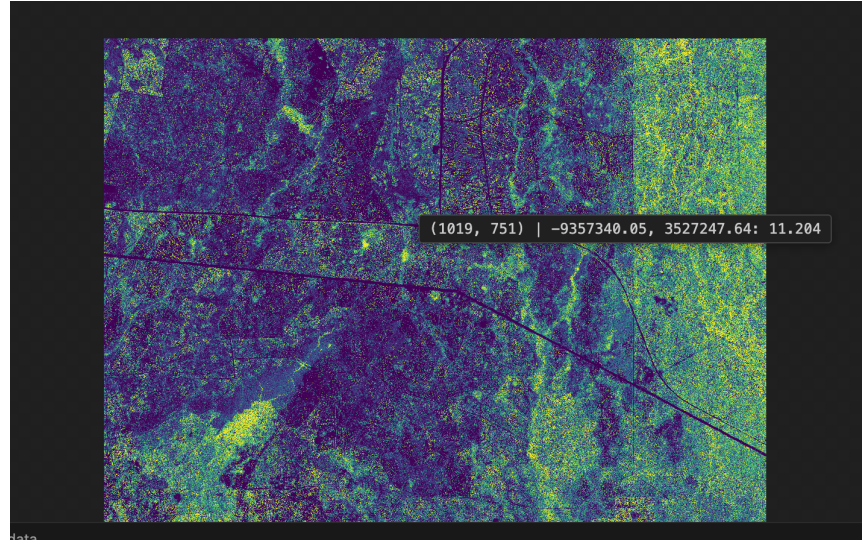


Fig. 1. A 1m digital surface model (DSM) of our testing area

While deploying A* directly on a DSM where the cell values represent heights above the imputed ground surface may be tempting – it is not a great idea. A* will indeed find the shortest path from start to goal, but this will be for a small $1\text{m} \times 1\text{m}$ line, entirely useless for the purposes of maximizing ground visibility from a low-flying aircraft. Further, the linear nature of height measurement does not really match the reality of the goal we are trying to accomplish. We don't actually care about vegetation height, we care about how much of the ground we can see.

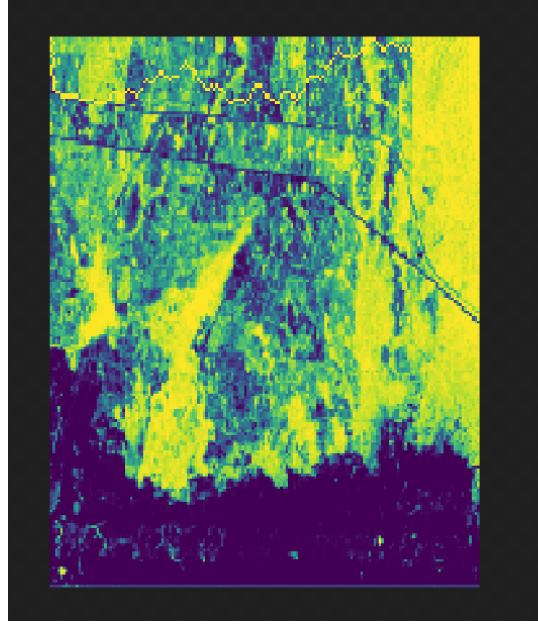


Fig. 2. An example of a constrained A* path on a DSM scaled and adjusted to represent surface visibility with darker areas being more visible, the path is highlighted in yellow

Similarly, vegetation at a height of $2m$ does not make the ground twice less visible than vegetation at a height of $1m$. $2m$ tall vegetation makes the ground almost entirely obstructed. In order to formulate our problem into one we can readily solve with standard pathfinding algorithms we can choose to describe each area by the proportion of ground/near ground points visible, rather than the absolute height. This gives us an acceptable metric of how much ground we can see in a given area. We also want some notion of a viewshed, that is – we don't really care about just some small $1m$ or $5m$ line, we care about what an observer from an aircraft can see. Usually, you could inform some of this using standard viewshed calculation algorithms, but since this is the U.S. Southeast and it's entirely flat, we can do away with that expensive operation for the most part and say you can probably see maybe $100 - 200m$ in each direction well. Since our 3D point cloud is now little more than a georeferenced 1-band image – we can accomplish this by defining a new matrix where each cell is defined in terms of a blur operation like the below for a given cell/node.

$$\bar{x}_{ij} = \frac{1}{|n(x_{ij})|} \sum_{y \in n(x_{ij})}^n y_i \quad (3)$$

Where $n(x)$ defines the neighborhood of x_{ij} , which amounts to a circular pass of radius r from x_{ij} .

Now we've addressed the problem of a lack of information about the state of surrounding cells/nodes, but we still haven't attempted to model the relationship between how much ground we can see and what the visibility is like in terms of how high the vegetation is. Since we determined that a small increase in vegetation height and thus a reduction in the proportion of ground points results in a significant reduction in ground/near-ground visibility, an exponentially scaling function seems like a natural choice. We used something of the form:

$$\frac{900}{e^{2e^x}} \quad (4)$$

This function has some desirable qualities, it is ≈ 1 when the proportion of ground points is close to 1 and scales rapidly to ≈ 120 as the proportion of ground points goes to 0. Better performance may be had by disincentivizing lower ground point concentrations even more highly using a piecewise function or similar.

3 Parallelization

Working with raw LiDAR data and performing calculations on large matrices makes our method quite computationally intense. Fortunately, due to the *embarrassingly parallel* nature of the task, it makes a perfect case for an application of data decomposition.

For a quick recap, our goal is to calculate flight lines, either from East to West or from North to South, across an entire input area. Notably, these flight lines are distanced from one another proportionally to a viewshed metric that we select depending on the parameters of the mission at hand (i.e. flight height, terrain blockage, etc.). This, in practice, means that each line is a completely separate calculation from one another (*embarrassingly parallel*). Furthermore, prior to calculating the flight line, we have to preprocess the raw LiDAR data such that it is compatible and optimal for our A* implementation. This task alone is computationally intense, as we first reduce the point-cloud representation to a 2D matrix representation, and then we aggregate surrounding pixels to be more representative of an area larger than the 1m granularity the initial reduction provides.

Initial sequential testing of our method confirmed the issues at hand; the entire script took multiple minutes to produce a single flight line. Naturally, we immediately looked to speed this up. To implement parallelization, we first created a wrapper class that would store our raw matrix data and some other important parameters for calculation. This class also has a method implemented that calculates a desirable flight line within the region denoted by the raw matrix. An overview of our implementation is as follows:

Algorithm 1 Parallelization

Require: Raw LIDAR data chunk from region, queue to return data to root

Ensure: Geodataframe representation of the optimal flight line put in queue

```

1: for  $i = 1$  to  $numblocks$  do                                     ▶ This loop is executed in parallel
2:   pathfinder  $\leftarrow$  FlightPath(LIDAR_data[i])
3:   geo_dataframe  $\leftarrow$  pathfinder.calculateFlightPath()
4:   queue.put(geo_dataframe)
5: end for

```

Algorithm 1 is responsible for the Master-Worker implementation that results in the root process receiving all flight lines by the end of all calculations.



Fig. 3. An example of a smoothed flight line within our interactive demo

Algorithm 2 calculateFlightPath

Require: No explicit input beyond initial class construction

Ensure: Optimal flight line for input region

```

1: for  $i = 1$  to  $row\_length$  do
2:   for  $j = 1$  to  $column\_length$  do
3:      $ground\_percent[i, j] \leftarrow Eq4(raw\_data[i, j])$ 
4:   end for
5: end for
6:  $path \leftarrow pyastar2d.astar\_path(ground\_percent)$ 
7: return: path

```

Algorithm 2 is responsible for preprocessing the LIDAR data and applying an implementation of A* to the resulting matrix of preprocessed data.

Upon the completion of the parallel section denoted in Algorithm 1, the root process will contain an (unsorted) list of all resulting flight lines. The final stage of the overall process is to sort the flight lines such that they are in order (Northernmost line to Southernmost line, Easternmost line to Westernmost line, etc.). Once the resulting flight lines are in an appropriate order, we link them all together to create one continuous flight line that alternates directions across the mapping.

4 Results

The algorithm was able to produce flight lines that appeared to avoid low ground visibility areas. Based on the formulation of the problem, we can be fairly sure that these flight lines indeed attempt to maximize ground visibility over their constrained area. The algorithm preforms best when run in an unconstrained environment, where it is free to move about to reach the goal – but this is often not the case in real world uses, where we typically only want a

flight line to vary by a few hundred meters or so. When tested in these environments, the algorithm naturally has a lot less leeway to find a better path, and so performance can suffer. Based on our evaluation - found by taking the sum of the total amount of scaled ground visible over the path and adjusting for path length, our method improved ground visibility by 32% over typical straight flight lines when tested on a random target area. Admittedly, it is quite difficult to test this type of algorithm empirically past a certain point save going on a flight run and seeing how it preforms. A sample flight line is available to be viewed here: <https://skyhog.vercel.app>

5 Conclusion and Further Discussion

Although our application of A* produced good results in a testing environment, we did not take into account factors that could hamper our accuracy. Chiefly, paved roadways. Roadways, which are often areas of low canopy density, can present an artificial area of low density we naturally do not want to follow for our purposes. The fact that they often run in fairly straight lines in cardinal directions can further complicate this. In *Figure 1*, you can clearly see where roadways and to a lesser concern natural gas lines (*center of image*) produce artificial pathways our algorithm would likely want to follow if given the opportunity. To rectify this, it would be sensible to incorporate roadways data into our weighting function to dis-incentivize following of these artificial pathways. Extensive roadway data is readily available in vector formats and can be used to mask a raster DSM.

More broadly, the proliferation of readily available, high resolution, LiDAR scans over the continental U.S. allows for a wealth of potential analysis. An application of similar methodologies used here though geared more towards optimizing for edge following would also likely be of interest to land managers, outdoorsmen and ecologists at large.